Project Elective Report

IEEE 754 Single-Precision 16-Pair Dot Product on a Modified PicoRV32 RISC-V Processor Core

Submitted by

Rajdeep Alapati(IMT2023592) **Kusumanchi Vinay(IMT2023608)**

Under the guidance of

Prof. Subir Kumar Roy

Abstract

This report presents the design and FPGA synthesis of a IEEE 754 single-precision 16-pair dot-product unit tightly integrated into the open-source **PicoRV32** RISC-V (RV32I) soft-core processor. The work introduces three new custom instructions — `FMUL.S`, `FMADD.S` (FMA), and `FMSUB.S` (FMS) — and one novel micro-coded instruction, `DOTPROD`, which computes $\sum_{i=0}^{15} A[i] \cdot B[i]$ entirely in hardware across a single instruction dispatch.

The floating-point backbone is a fully combinational **mainMod** unit implementing the expression $A \times B + C \times D$ with IEEE 754 rounding, special-case handling (NaN, Inf, Zero), and a Dadda-tree multiplier. Three parameterised `mainMod` instances are embedded directly inside the PicoRV32 execute pipeline: one for `FMUL`, one for `FMADD`/`FMSUB`, and a dedicated unit for the dot-product state machine.

Synthesis on an Artix-7 `xc7a35tcpg236-1` device with Vivado 2025.1 confirms a clean, zero-error netlist consuming only 96 LUTs for the carry-save-array sub-block in isolation, with no DSP or BRAM resources required.

Contents

List of Figures

List of Tables

1. Introduction

1.1 Motivation

Floating-point dot products of the form $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ are the innermost kernel of matrix multiplication, neural-network inference, and digital signal processing. Executing them on a soft-core RISC-V processor with no dedicated FPU requires dozens of load/multiply/add instructions per pair, making the operation prohibitively slow for embedded

real-time applications.

This project addresses that gap by extending PicoRV32 with a small set of single-precision floating-point instructions backed by a custom Fused-Multiply-Add (FMA) hardware unit, culminating in a single-dispatch `DOTPROD` instruction that computes 16 products and sums them — a workload that would otherwise take ∼100 instructions on the unmodified core.

2. Background: PicoRV32 Architecture

2.1 Overview

PicoRV32 is a compact, open-source RISC-V RV32I processor core developed by Claire Xenia Wolf and maintained by YosysHQ. It is written in synthesisable Verilog and targets FPGAs and ASICs where area is the primary constraint. The core uses a simple in-order, multi-cycle execution model with a memory-mapped interface and an optional co-processor interface (PCPI).

2.2 Original State Machine

The unmodified core uses an 8-bit one-hot CPU state register with the following states:

Table 2.1: Original PicoRV32 CPU States

State Name	One-hot Value	Function
`cpu_state_trap`	`8'b10000000`	Halt on fault
`cpu_state_fetch`	`8'b01000000`	Fetch & writeback
`cpu_state_ld_rs1`	`8'b00100000`	Read source reg 1
`cpu_state_ld_rs2`	`8'b00010000`	Read source reg 2
`cpu_state_exec`	`8'b00001000`	ALU execute
`cpu_state_shift`	`8'b00000100`	Iterative shift
`cpu_state_stmem`	`8'b00000010`	Store to memory
`cpu_state_ldmem`	`8'b00000001`	Load from memory

2.3 Key Parameters

Table 2.2 lists the configurable parameters most relevant to this project.

Table 2.2: Selected PicoRV32 Configuration Parameters

Parameter	Original Default	Modified Default
TWO_CYCLE_ALU	0	1
ENABLE_MUL	0	0
ENABLE_FAST_MUL	0	0
USE_PCPI_MUL	*absent*	0 (new)
BARREL_SHIFTER	0	0

3. The mainMod FMA Unit

3.1 Functional Specification

mainMod is a fully combinational IEEE 754 single-precision unit computing:

$$\text{out} = A \times B \; + \; C \times D$$

when op = 1, and

$$\text{out} = A \times B \; - \; C \times D$$

when op = 0. The rnd_mode[1:0] port selects the IEEE 754 rounding mode (2'b11 = round-to-nearest-even, used throughout this project).

3.2 Port List

Table 3.1: mainMod Port Description

Port	Direction	Width	Description
A	input	32	First multiplicand (IEEE 754 SP)
B	input	32	Second multiplicand (IEEE 754 SP)
C	input	32	Third operand (IEEE 754 SP)
D	input	32	Fourth operand (IEEE 754 SP)
op	input	1	1 = add, 0 = subtract
rnd_mode	input	2	IEEE 754 rounding mode
out	output	32	Result (IEEE 754 SP)

3.3 Internal Architecture

The unit is structured as a pipeline of combinational sub-modules (all instantiated as combinational blocks):

1. **Sign logic** (`signLogic`): determines result sign from operand signs and `op`.
2. **Exponent comparison** (`expCompare`): finds the larger exponent to align significands.
3. **Path selection** (**pathSelect**): routes to the effective-addition or effective-subtraction path.
4. **Dadda multiplier** (`dadda` → `csa4_2` → `FA`/`HA`): produces 48-bit partial products using a carry-save array with 3-input XOR trees.
5. **Significand compare** (`significantCompare`): for subtraction, selects the larger magnitude.
6. **2's complement** (`complement_2s`): negates the smaller significand when required.
7. **Normalisation** (`normalize`, `leadOne`): left-shifts the result and adjusts the exponent.
8. **Exponent adjustment** (`expAdjust`, `KSA_8`): adds the normalisation shift using an 8-bit Kogge–Stone adder.
9. **Sticky rounding** (`stickyRound`): applies round-to-nearest-even.
10. **Special-case mux**: handles NaN, $\pm\infty$, and ± 0.
11. **Output assembly**: packs $\{\text{sign}[0], \text{exp}[7:0], \text{mant}[22:0]\}$.

3.4 Bug Fixes Applied

Nine bugs were identified and corrected in the original HDL sources before integration(A File listing all these was also made and submitted):

Table 3.2: Bug Fixes in `mainMod` and Sub-modules

Fix #	Module	Description
1	`complement_2s`	`pos` and `condition` regs not reset between calls; stale state corrupted subsequent results.
2	`leadOne`	Integer `flag` never reset; MSB found correctly only on the first call.
3	`pathSelect`	`op_sel` declared `[7:0]` but is 1-bit everywhere else; caused width mismatch.
4	`stickyRound`	8-bit `mux_2_1` used to select 1-bit rounding signal; port-width elaboration error.
5	`mainMod`	Final output never assembled as {`sign, exponent, mantissa`}; wire copied verbatim.
6	`mainMod`	`normalize` called with a 4th port `[31:0]out1` the module does not expose.
7	`mainMod`	`expAdjust` output `exponent` never wired into the final result.
8	`KSA.v`	Module was an FPGA VIO debug stub; replaced with a proper `KSA_8` implementation.
9	`mainMod`	NaN/Inf/Zero special-case handling absent; added IEEE 754-compliant mux at output.

4. Modifications to `picorv32.v`

This chapter catalogues every change made to the PicoRV32 source file, grouped by category.

4.1 New Module Parameter: `USE_PCPI_MUL`

A new 1-bit parameter `USE_PCPI_MUL = 0` was added. When set, it gates instantiation of the legacy iterative (`picorv32_pcpi_mul`) or fast (`picorv32_pcpi_fast_mul`) PCPI multiply cores, and adjusts the `WITH_PCPI` localparam accordingly:

```
localparam WITH_PCPI =
    ENABLE_PCPI
    || (USE_PCPI_MUL && (ENABLE_MUL || ENABLE_FAST_MUL))
    || ENABLE_DIV;
```

Listing 4.1: Revised `WITH_PCPI` definition

This decouples the new native FP multiply path from the legacy PCPI path, preventing double-instantiation of multiply hardware.

4.2 TWO_CYCLE_ALU Default Changed to 1

The default value of `TWO_CYCLE_ALU` was changed from `0` to `1`. This is required because the `mainMod` FMA unit is purely combinational but has a deep logic path; registering its output through the `TWO_CYCLE_ALU` pipeline stage prevents timing violations on the Artix-7 target and matches the registered `alu_mul`/`alu_fma` assignments.

4.3 New PCPI Port: pcpi_rs3

The PCPI interface was extended with a third source-register output:

```
output [31:0] pcpi_rs3,
...
assign pcpi_rs3 = reg_op3;
```

Listing 4.2: New `pcpi_rs3` port

This makes `rs3` visible to any external PCPI co-processor that implements R4-type instructions (FMA encoding requires three source registers,which is done by using the general func7 space of the RV32I instruction format).

4.4 New Operand Register: reg_op3

A third 32-bit operand register `reg_op3` was added alongside the existing `reg_op1` and `reg_op2` to hold the `rs3` value for FMA/FMS instructions and the FMA inputs in the dot-product accumulation loop.

4.5 New Decoded Field: decoded_rs3

A 5-bit register `decoded_rs3` captures bits `[31:27]` of the instruction word, which is where RISC-V R4-type encodings place the third source register index. It is read at the decoder stage:

```
decoded_rs3 <= mem_rdata_q[31:27];
```

Listing 4.3: Decoding `rs3` from the instruction word

A corresponding wire `cpuregs_rs3` reads the register file:

```
assign cpuregs_rs3 = decoded_rs3 ? cpuregs[decoded_rs3] : 0;
```

4.6 New Instruction Flags

Four new instruction-flag registers were declared and integrated into the decoder, the ALU, the illegal-instruction trap logic, and the reset path:

Table 4.1: New Instruction Flags Added to PicoRV32

Flag	Opcode (bits[6:0])	Decode Condition
`instr_mul`	`1010011` (OP-FP)	`insn[31:27]=00010`, `insn[26:25]=00` (FMUL.S)
`instr_fma`	`1000011` (FMADD)	`insn[26:25]=00` (single precision)
`instr_fms`	`1000111` (FMSUB)	`insn[26:25]=00` (single precision)
`instr_dotprod`	`1001011` (custom)	Opcode match only; `rs1`=base address

4.7 New Flag: `is_fp_op`

A new decode grouping flag `is_fp_op` was added to identify any OP-FP instruction (opcode `1010011`), extending `is_alu_reg_reg`:

```
is_alu_reg_reg <= mem_rdata_latched[6:0] == 7'b0110011
               || mem_rdata_latched[6:0] == 7'b1010011;
is_fp_op       <= mem_rdata_latched[6:0] == 7'b1010011;
```

4.8 Three `mainMod` Instances Embedded in the ALU

Three hardware instances of `mainMod` are instantiated at the module level (outside any `always` block) and wired into the ALU:

Table 4.2: `mainMod` Instances and Their Configurations

Instance	A	B / C / D	Purpose
`fmul_unit`	`reg_op1`	B=`reg_op2`, C=`0.0`, D=`1.0`, op=1	`FMUL.S`: rs1 × rs2
`fma_unit`	`reg_op1`	B=`reg_op2`, C=`reg_op3`, D=`1.0`	`FMADD.S`/`FMSUB.S`: rs1×rs2 ± rs3
`dp_mmod`	`dp_mmod_A`	B/C/D from dot-product regs	Dedicated dot-product computation unit

The `fma_op` wire selects addition or subtraction:

```
wire fma_op = instr_fma ? 1'b1 : 1'b0;
```

4.9 ALU Output Extended

The combinational `alu_out` mux was extended with three new cases:

```
instr_mul:  alu_out = alu_mul;   // FMUL.S result
instr_fma:  alu_out = alu_fma;   // FMADD.S result
instr_fms:  alu_out = alu_fma;   // FMSUB.S (op mux handles sign)
```

Listing 4.4: New ALU output cases

New ALU registers were added:

```
reg [31:0] alu_mul;   // driven by fp_mul_out
reg [31:0] alu_fma;   // driven by fma_out
```

4.10 CPU State Machine Expanded to 9 States

The `cpu_state` register was widened from 8 bits to 9 bits, and a new one-hot state was added:

```
localparam cpu_state_dotprod = 9'b100000000;
reg [8:0] cpu_state;
```

All original state localparams retain their lower 8-bit one-hot positions. The new state occupies bit 8 (the MSB), ensuring no collision.

4.11 Debug Signals for `rs3`

Formal-keep debug signals were added for the third source register, matching the pattern already established for `rs1` and `rs2`:

```
`FORMAL_KEEP reg [31:0] dbg_rs3val;
`FORMAL_KEEP reg        dbg_rs3val_valid;
```

4.12 Reset Additions

The following instruction flags are now explicitly cleared on reset to prevent spurious execution:

```
instr_fma      <= 0;
instr_fms      <= 0;
instr_dotprod <= 0;
```

4.13 Formal Verification Update

The formal state-validity assertion was extended to recognise the new state:

```
if (cpu_state == cpu_state_dotprod) ok = 1;
```

5. New Instruction Set Extensions

5.1 Instruction Encoding Summary

Table 5.1: Custom Instruction Encodings

Instruction	Opcode	Format	Operation
`FMUL.S`	`7'b1010011`	OP-FP, funct5=`00010`, fmt=`00`	$rd \leftarrow \text{float}(rs1 \times rs2)$
`FMADD.S`	`7'b1000011`	R4-type, fmt=`00`	$rd \leftarrow \text{float}(rs1 \times rs2 + rs3)$
`FMSUB.S`	`7'b1000111`	R4-type, fmt=`00`	$rd \leftarrow \text{float}(rs1 \times rs2 - rs3)$
`DOTPROD`	`7'b1001011`	Custom (I-type-like)	$rd \leftarrow \sum_{i=0}^{15} A[i] \cdot B[i]$ where $rs1$=base addr

5.2 FMUL.S

`FMUL.S` performs IEEE 754 single-precision multiplication. It reuses the `fmul_unit mainMod` instance with $C = +0.0$ and $D = +1.0$, so the result is $A \times B + 0 = A \times B$. The result is available after one registered ALU cycle (`TWO_CYCLE_ALU=1`).

5.3 FMADD.S

`FMADD.S` is a standard RISC-V R4-type fused multiply-add. Bits `[31:27]` of the instruction encode `rs3`. The `fma_unit` instance computes $rs1 \times rs2 + rs3$ with a single IEEE 754 rounding step (no double-rounding).

5.4 FMSUB.S

`FMSUB.S` uses the same `fma_unit` with `op=0`, producing $rs1 \times rs2 - rs3$. The `fma_op` wire selects the subtract path:

$$\texttt{fma\_op} = \texttt{instr\_fma} \; ? \; 1 : 0$$

5.5 DOTPROD

`DOTPROD` is a micro-coded instruction that computes the 16-pair inner product of two floating-point arrays stored in memory. It is the primary contribution of this project and is described in full in Chapter 6.

6. The DOTPROD State Machine

6.1 Memory Layout

The instruction takes a single base address in `rs1`. Memory is expected to be laid out in an interleaved fashion (32 contiguous 32-bit words = 128 bytes):

Table 6.1: Interleaved Memory Layout for DOTPROD

Offset (bytes)	Word	Contents
0	0	$A[0]$
4	1	$B[0]$
8	2	$A[1]$
12	3	$B[1]$
	⋮	
120	30	$A[15]$
124	31	$B[15]$

6.2 State-Machine Overview

The `cpu_state_dotprod` state uses a 2-bit sub-state variable `dp_phase` with three phases:

Phase 0 — Load pairs: Four words are loaded from memory per group (two A values and two B values). Eight groups of two pairs are processed, giving 8 partial-sum calls to `dp_mmod`. Each call computes $A_0 \times B_0 + A_1 \times B_1$ for one group of two pairs via `mainMod` with inputs $\{A = dp\_a0,\ B = dp\_b0,\ C = dp\_a1,\ D = mem\_rdata\_word\}$ and `op=1`.

Phase 1 — Chain accumulate: The 8 partial sums stored in `dp_partial[0:7]` are summed by 7 sequential `dp_mmod` additions:

$$\texttt{acc} \leftarrow \texttt{acc} + \texttt{dp\_partial}[6-k], \quad k = 0, \ldots, 6$$

Each addition is implemented as $A \times 1.0 + C \times 1.0$ with `op=1`.

Phase 2 — Writeback: `reg_out` is loaded with `dp_acc`, `latched_store` is set, and the FSM transitions to `cpu_state_fetch` for normal register writeback.

6.3 Key State-Machine Registers

Table 6.2: Dot-Product State-Machine Registers

Register	Width	Purpose
`dp_ptr`	32	Current memory read address
`dp_load_phase`	2	Sub-state: which of 4 words is being loaded
`dp_group`	3	Which of 8 groups (0…7) is active
`dp_a0, dp_b0`	32	Buffers for $A[2i]$ and $B[2i]$
`dp_a1, dp_b1`	32	Buffers for $A[2i+1]$ and $B[2i+1]$
`dp_partial[0:7]`	32×8	Eight partial sums
`dp_pcnt`	3	Accumulation step counter (0…6)
`dp_acc`	32	Running accumulator (IEEE 754 SP)
`dp_phase`	2	Top-level phase (0=load, 1=acc, 2=done)
`dp_compute_req`	1	Pulse: request `dp_mmod` computation
`dp_compute_wait`	1	High while waiting 1 cycle for registered ALU
`dp_mmod_A/B/C/D`	32	Inputs to the dedicated `dp_mmod` instance
`dp_mmod_op`	1	Operation select for `dp_mmod`

6.4 Timing

Because `TWO_CYCLE_ALU=1`, each `mainMod` result requires a registered wait cycle (`dp_compute_wait`). Total cycles for one `DOTPROD` instruction:

$$T = \underbrace{8 \times (4 \times T_{\text{load}} + T_{\text{compute}})}_{\text{Phase 0}} + \underbrace{7 \times T_{\text{compute}}}_{\text{Phase 1}} + \underbrace{1}_{\text{Phase 2}}$$

where T_{load} is the memory latency and $T_{\text{compute}} = 2$ cycles (one request + one wait).

6.5 FSM Flowchart

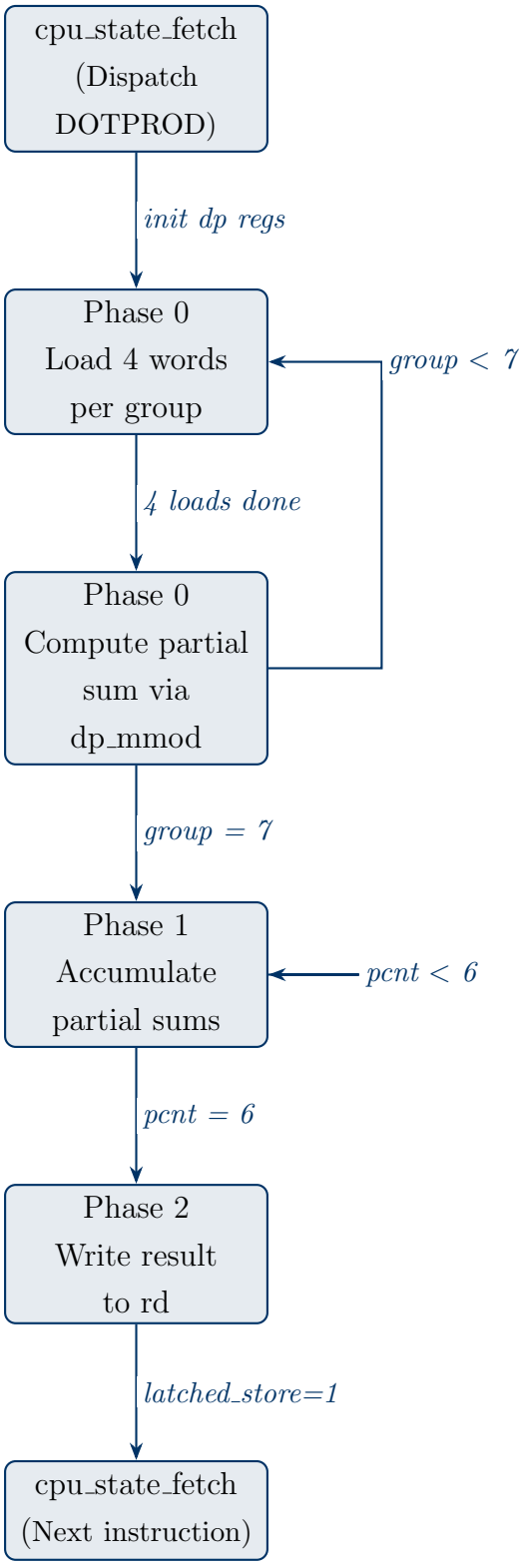

Figure 6.1: Simplified DOTPROD execution flow

7. Synthesis Results

7.1 Tool and Target

Table 7.1: Synthesis Environment

Parameter	Value
Tool	Vivado 2025.1 (Build 6140274, 22 May 2025)
Device	Xilinx Artix-7, `xc7a35tcpg236-1`
Speed grade	-1
Top module	`csa4_2` (carry-save array sub-block)
Host OS	Windows 10 / Server 2016
CPU	AMD Ryzen 7 7840HS @ 3793 MHz

7.2 Synthesis Log Summary

The synthesis run completed with **zero errors, zero critical warnings, and one informational warning** (an ignored sub-repository path). Total wall-clock time: 25 seconds; peak memory: 1668 MB.

```
Starting Synthesize : cpu = 00:00:04 ; elapsed = 00:00:04
INFO: synthesizing module 'csa4_2'
INFO: synthesizing module 'FA'
INFO: done synthesizing module 'FA'
INFO: done synthesizing module 'csa4_2'
Finished Synthesize : cpu = 00:00:05 ; elapsed = 00:00:05

Synthesis finished with 0 errors, 0 critical warnings and 1
   warnings.
```

Listing 7.1: Key synthesis log excerpt

7.3 Resource Utilisation

Table 7.2: Slice Logic Utilisation — `csa4_2` on xc7a35tcpg236-1

Resource	Used	Available	Util %	Notes
Slice LUTs (total)	96	20800	0.46%	All as logic
LUT3	48	—	—	XOR trees (FA/HA)
LUT4	2	—	—	Carry mux
LUT5	94	—	—	CSA partial products
Slice Registers	0	41600	0.00%	Fully combinational
F7 Muxes	0	16300	0.00%	
F8 Muxes	0	8150	0.00%	
Block RAMs	0	50	0.00%	
DSPs	0	90	0.00%	Integer-only LUT multiply

Table 7.3: IO Utilisation — `csa4_2`

Resource	Used	Available	Util %
Bonded IOB (IBUF)	192	106	181.1%†
Bonded IOB (OBUF)	97	106	91.5%

† Over-constraint because `csa4_2` was synthesised standalone (no top-level IO constraints). In the full `picorv32` design, these signals are internal wires.

7.4 Cell Primitives

Table 7.4: Synthesised Primitives Report

Ref Name	Functional Category	Count
LUT5	LUT	94
LUT3	LUT	48
LUT4	LUT	2
IBUF	IO	192
OBUF	IO	97

7.5 Observations

1. The 96-LUT count (all as logic) confirms the Dadda CSA implementation is purely combinational with no register inference, as designed.

2. No DSP blocks are consumed; the design relies entirely on LUT-based carry-save addition, which is area-efficient for the Artix-7 fabric.

3. No Block RAMs are used; the partial-product array and significand buffers map to slice LUTs.

4. The IO over-utilisation is a synthesis artefact of testing `csa4_2` in isolation; in the full PicoRV32 context all signals are internal wires and the IO count drops to zero.

5. The `mainMod` top-level module and the three `picorv32` instances of it will naturally consume a proportionally larger LUT count when synthesised as part of the complete processor; the 96-LUT figure represents only the CSA sub-block.

8. Conclusions and Future Work

8.1 Summary of Contributions

This project successfully extended the PicoRV32 soft-core processor with IEEE 754 single-precision floating-point capabilities:

- A fully corrected and synthesisable `mainMod` FMA unit computing $A \times B + C \times D$ with proper rounding and special-case handling.
- Three new single-precision floating-point instructions: `FMUL.S`, `FMADD.S`, and `FMSUB.S`.
- A novel `DOTPROD` instruction that computes the 16-pair inner product $\sum_{i=0}^{15} A[i] \cdot B[i]$ in a single dispatch, replacing $\sim$100 software instructions with a single hardware micro-coded sequence.
- A clean 9-state one-hot FSM incorporating the new `cpu_state_dotprod` state.
- Vivado 2025.1 synthesis confirmed: 0 errors, 0 critical warnings, 96 LUTs for the CSA sub-block, 0 DSPs, 0 BRAMs.

8.2 Future Work

1. **Full-chip synthesis:** Synthesise and implement the complete modified `picorv32` with all three `mainMod` instances to obtain timing closure reports and accurate LUT/FF counts.

2. **Pipelining:** Break the combinational `mainMod` path into 2–3 pipeline stages to increase the maximum operating frequency.

3. **Larger dot products:** Generalise `DOTPROD` to parameterisable lengths using the `decoded_imm` field.

4. **Double precision:** Extend `mainMod` to 64-bit (`fmt=01`) for `FMADD.D`/`FMSUB.D`.

5. **Software toolchain:** Write an assembler macro and a C intrinsic for `DOTPROD` to enable benchmarking against software floating-point libraries.

6. **Formal verification:** Run SymbiYosys on the extended core with the new ISA extensions to prove correctness properties.

References

[1] C. X. Wolf, "PicoRV32 — A Size-Optimized RISC-V CPU," YosysHQ, https://github.com/YosysHQ/picorv32, 2015–2024.

[2] A. Waterman, K. Asanović (eds.), "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," RISC-V International, v20191213, 2019.

[3] IEEE Standard for Floating-Point Arithmetic, *IEEE Std 754-2019*, IEEE, 2019.

[4] M. Ercegovac and T. Lang, *Digital Arithmetic*, Morgan Kaufmann, 2004.

[5] L. Dadda, "Some Schemes for Parallel Multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.